Compte rendu

B-1 (/5pts) Créer la classe graphe et rechercher l'algorithme en l'intégrant comme méthode de la classe. Tester votre programme pour la liste d'adjacence des successeurs suivante :

adj = {'A':{'B':15,'C':4},'B':{'E':5},'C':{'E':11,'D':2},'D':{'E':3},'E':{}}

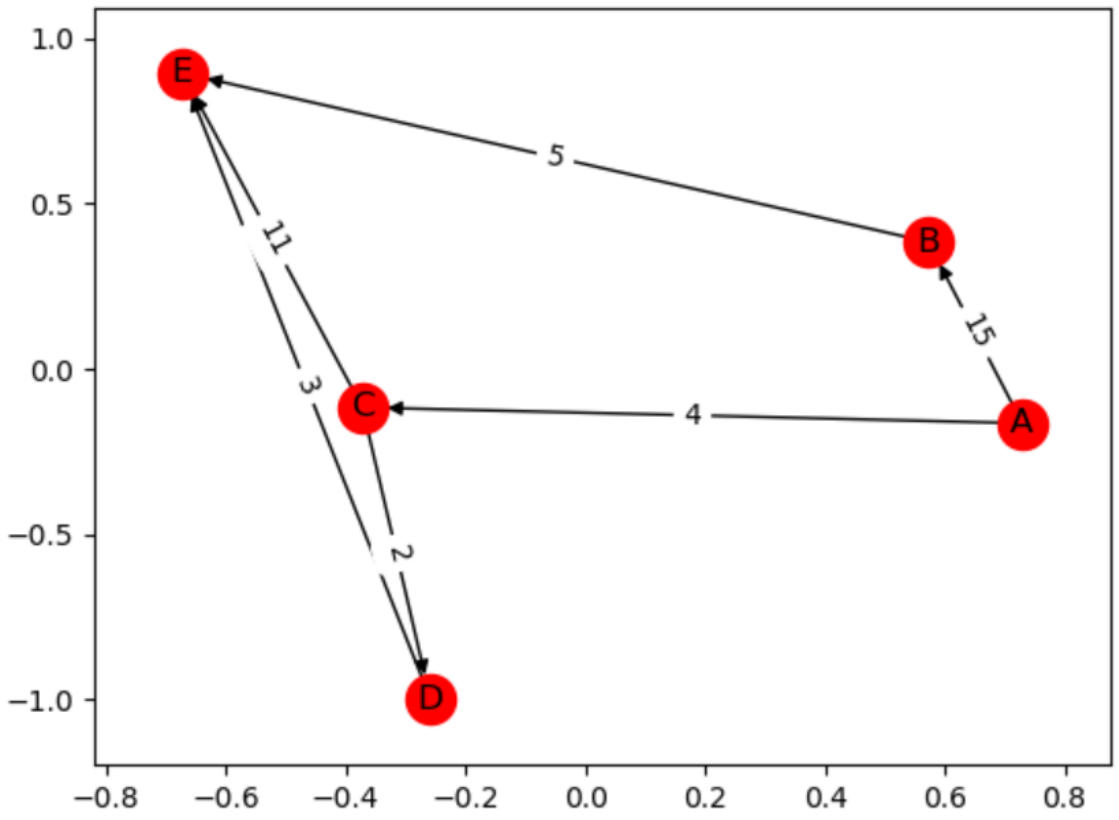
```
from PileFile import PileFile
import networkx as nx
import matplotlib.pyplot as plt

class graphe:

    def __init__(self, mat_adj):
        self.__graphe = mat_adj

    def dijkstra(self, sommet_depart, sommet_arrivee):
        observe = PileFile()
        visite = []
        chemin = []
        dejavu = []
        distance = {sommet_depart: 0}

        observe.enfiler(sommet_depart)

        while not observe.estVide():
            sommet = observe.defiler()
            for voisin in self.__graphe[sommet]:
                observe.enfiler(voisin)

                nouvelle_distance = distance[sommet] + self.__graphe[sommet][voisin]
                if voisin not in distance or nouvelle_distance < distance[voisin]:
                    distance[voisin] = nouvelle_distance
                    dejavu.append(voisin)
                    visite.append(sommet)

        chemin.append(sommet_arrivee)
        for i in range(len(dejavu) - 1, -1, -1):
            if chemin[-1] == dejavu[i]:
                chemin.append(visite[i])
        if sommet_depart not in chemin:
            chemin.append(sommet_depart)
        chemin.reverse()

        return chemin, distance[sommet_arriveeS]
```

```python
    def dessiner(self):
        G = nx.DiGraph()
        weighted_edges = []
        for node_p, nodes_s in self.__graphe.items():
            for node_s, weight in nodes_s.items():
                weighted_edges.append((node_p, node_s, weight))
        G.add_weighted_edges_from(weighted_edges)
        pos = nx.spring_layout(G, seed=2)
        nx.draw_networkx(G, pos)
        labels = nx.get_edge_attributes(G, 'weight')
        nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
        plt.show()


if __name__ == "__main__":
    adj = {'A':{'B':15,'C':4},'B':{'E':5},'C':{'E':11,'D':2},'D':{'E':3},'E':{}}
    g = graphe(adj)
    chemin, distance = g.dijkstra('A', 'E')
    print("Chemin le plus court :", chemin)
    print("Distance totale :", distance)
    g.dessiner()
```

affichage de la console :

```
*** Distant Python engine is active ***
>>>
*** Console de processus distant Réinitialisée ***
Chemin le plus court : ['A', 'C', 'D', 'E']
Distance totale : 9
```

Figure générée :



B-2 (/1pt) Ajouter au code précédent, une variable booléenne permettant la détection d'un cycle (retour à un sommet déjà visité) en renvoyant une variable booléenne dans la méthode Dijsktra().

Ajouts de bouts de code :

```
def dijkstra(self, sommet_depart, sommet_arrivee):
    observe = PileFile()
    visite = []
    chemin = []
    dejavu = []
    distance = {sommet_depart: 0}
    cycle_detecte = False
```

ajout de la variable

```
while not observe.estVide():
    sommet = observe.defiler()

    if sommet in visite:
        cycle_detecte = True
```

verifie sur le sommet a déjà été visité

```
return chemin, distance[sommet_arrivee], cycle_detecte
```

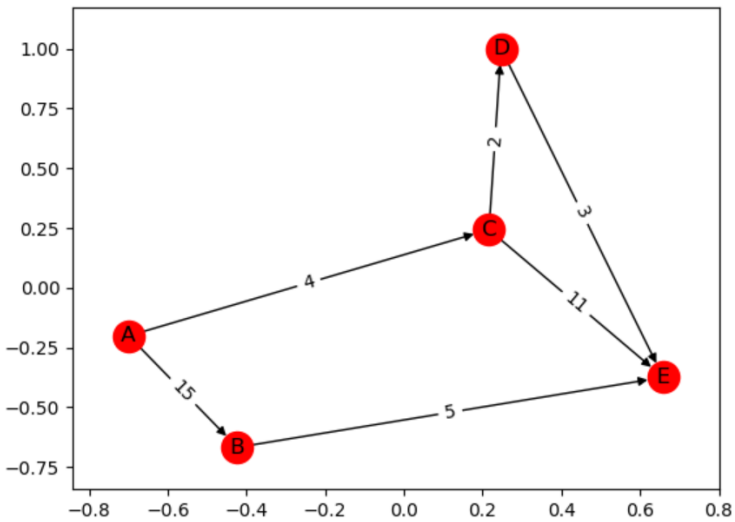
ajout de cycle_detecte pour que ça renvoie le booléen

```
if __name__ == "__main__":
    adj = {'A':{'B':15,'C':4},'B':{'E':5},'C':{'E':11,'D':2},'D':{'E':3},'E':{}}
    g = graphe(adj)
    chemin, distance, cycle = g.dijkstra('A', 'E')
    print("Chemin le plus court :", chemin)
    print("Distance totale :", distance)
    print("Cycle détecté :", cycle)
    g.dessiner()
```

Ajout dans le main

B-3 (/2pts) Dessiner ce graphe à l'aide de la fonction fournie et donner le chemin le plus court entre A et E.

Graphe :



Chemin le plus court :

```
Chemin le plus court : ['A', 'C', 'D', 'E']
Distance totale : 9
Cycle détecté : False
```

B-4 (/2pts) A l'aide de la carte suivante, déterminer la liste d'adjacence des successeurs de Z vers L (une distance ne doit être reportée qu'une fois dans la liste) et appliquer l'algorithme de Djiskstra pour aller Bayonne à Lyon pour déterminer le chemin le plus court.

Liste d'adjacence :
adj1 =
{'Z':{'B':185,'T':296},'B':{'R':205,'T':245},'R':{'C':179,'T':200},'C':{'L':164,'P':332},'T':{'P':242},'P':{'V':203,'M':168},'V':{'L':102,'M':213},'M':{},'L':{}}

Implémentation dans le code :

```
if __name__ == "__main__":
    adj = {'A':{'B':15,'C':4},'B':{'E':5},'C':{'E':11,'D':2},'D':{'E':3},'E':{}}
    adj1 = {'Z':{'B':185,'T':296},'B':{'R':205,'T':245},'R':{'C':179,'T':200},'C':{'L':164,'P':332},'T':{'P':242},'P':{'V':203,'M':168},'V':{'L':102,'M':213},'M':{},'L':{}}
    g = graphe(adj1)
    chemin, distance, cycle = g.dijkstra('Z', 'L')
    print("Chemin le plus court :", chemin)
    print("Distance totale :", distance)
    print("Cycle détecté :", cycle)
    g.dessiner()
```

Affichage de la console :

```
Chemin le plus court : ['Z', 'B', 'R', 'C', 'L']
Distance totale : 733
```

C-1 (/0.5pt) Déterminer en langage Python les coordonnées GPS de votre domicile et de votre lycée.

```
if __name__ == "__main__":
    coord_domicile = coord_GPS_addresse("14 rue de la biche", "Chelles")
    print("Coordonnées GPS du domicile :", coord_domicile)
    coord_lycee = coord_GPS_addresse("32 avenue de l'Europe", "Chelles")
    print("Coordonnées GPS du lycée :", coord_lycee)
```

Affichage de la console :

```
Coordonnées GPS du domicile : (48.8906761, 2.6075438)
Coordonnées GPS du lycée : (48.882752, 2.61227)
```

C-2 (/0.5pt) Afficher et copier le graphe de la zone géographique incluant le départ et l'arrivée. (Attention le temps de traitement peut être long selon la distance).

Modification de la fonction graphe pour ne pas afficher de trajet :

```
def graphe(adresse_depart,Ville_D,adresse_arrivee,Ville_A,type_graph):
    origine_GPS=coord_GPS_addresse(adresse_depart,Ville_D)
    destination_GPS=coord_GPS_addresse(adresse_arrivee,Ville_A)
    point = ((origine_GPS[0]+destination_GPS[0])/2,(origine_GPS[1]+destination_GPS[1])/2)
    dist=distance_2pGPS(origine_GPS,destination_GPS)/1.5
    G = ox.graph_from_point(point, distance=dist, network_type=type_graph)
    #origine=ox.get_nearest_node(G, origine_GPS)
    #destination=ox.get_nearest_node(G, destination_GPS)
    #route = nx.shortest_path(G, origine, destination)
    #ox.plot_graph_route(G,route,fig_height=10, fig_width=10, show=True, use_geom=True, close=False, route_color='green' ,orig_dest_node_size=100)
    ox.plot_graph(G, fig_height=10, fig_width=10, show=True)
    return G
```

Appel :

```
G = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "all")
```

graphe :

C-3(/0.5pt) Afficher et copier le graphe avec l'itinéraire le plus court pour les deux moyens de locomotion.

Appel pour les transports en commun :

```
G_drive_service, route_drive_service = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "drive_service")
```

Graphe :

Appel pour à pied :

```
G_walk, route_walk = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "walk")
```

Graphe :

C-4 (/0.5pt) Copier le contenu de la variable route utilisé dans le programme.

Ajout d'un print pour afficher le contenu de la variable route_drive_service :

```
G_drive_service, route_drive_service = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "drive_service")
print("Contenu de la variable route_drive_service pour le trajet en transports :", route_drive_service)
```

Affichage de la console :

```
Contenu de la variable route_drive_service pour le trajet en transports : [266608632, 266604653, 2304319952, 266595256, 267665162, 266594466, 266594468, 266594469,
 267665291, 267665350, 2639635875, 2639635862, 503849824, 267665342, 267665636, 267663374, 267663377, 274006099, 8485986677, 273967831, 8220131461, 273965572, 273965573,
 2430642774, 2430642779, 273963800, 267229858, 267228785]
```

Ajout d'un print pour afficher le contenu de la variable route_walk :

```
G_walk, route_walk = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "walk")
print("Contenu de la variable route_walk pour le trajet à pied :", route_walk)
```

Affichage de la console :

```
Contenu de la variable route_walk pour le trajet à pied : [266608632, 4038894388, 266604653, 9269320028, 9269320029, 2304319968, 503849910, 2639635865, 2639635862,
 503849824, 267668093, 503849770, 267669140, 267669139, 267669515, 8626147917, 267672168, 503848919, 267671861, 267671862, 267672672, 503848867, 7101512689, 7101512694,
 7101512733, 7101512725, 483229917, 483229907, 2315899933, 2315899934, 2315899935, 6183808801]
```

C-5 (/0.5pt) Générer une page HTML permettant de visualiser le parcours en véhicule sur une carte interactive.

```
G_drive, route_drive = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "drive")
graphique_html(G_drive, route_drive)
```

Page html :

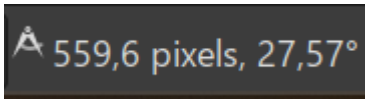
C-6 (/1pt) Calculer l'orientation par rapport au Nord entre votre point de départ et d'arrivée en langage Python. Vérifier la valeur trouvée sur le graphe récupéré en question C-2 à l'aide du logiciel GIMP ou d'un rapporteur.

```
coord_domicile = coord_GPS_addresse("14 rue de la biche", "Chelles")
coord_lycee = coord_GPS_addresse("32 avenue de l'Europe", "Chelles")
orientation = orientation(coord_domicile, coord_lycee)
print("Orientation par rapport au Nord :", orientation, "degrés")
```

Affichage de la console :

```
Orientation par rapport au Nord : 201.41305785137013 degrés
```

Verification avec GIMP :



On ajouter 180 degrés pour avoir la même orientation : 27 + 180 = 207. La valeur correspond a peu près à ce que pyhton nous a donné.

C-7-1(/3pts) Trouver le code python de l'algorithme permettant le calcul de la distance totale du parcours en langage Python. En observant le contenu du dictionnaire G.adj, proposer une autre solution pour déterminer la distance totale du parcours, on s'intéressera à la récupération de la distance de chaque arrête contenu dans cette variable.

```python
# Algorithme de calcul de distance à rechercher par les élèves en TP
def distance_totale(G, route):
    distance = 0
    for sommet in range(len(route) - 1):
        sommet_suivant = sommet + 1
        lat1 = G.node[route[sommet]]['y']
        lon1 = G.node[route[sommet]]['x']
        lat2 = G.node[route[sommet_suivant]]['y']
        lon2 = G.node[route[sommet_suivant]]['x']
        d = distance_2pGPS((lat1, lon1), (lat2, lon2))
        distance += d
    return distance
```

Version avec le dictionnaire G.adj :

```python
def distance_totale_adj(G, route):
    distance = 0
    for i in range(len(route) - 1):
        sommet_actuel = route[i]
        sommet_suivant = route[i + 1]
        arrete = list(G.adj[sommet_actuel][sommet_suivant].values())[0]
        longueur_arrete = arrete["length"]
        distance += longueur_arrete
    return distance
```

C-7-2(/1pt) Programmer et tester votre fonction sur votre parcours. Donner la valeur de la distance desdeux itinéraires.

```python
G_drive_service, route_drive_service = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "drive_service")

distance_gps = distance_totale(G_drive_service, route_drive_service)
print("Distance totale (méthode GPS) :", distance_gps, "mètres")

distance_adj = distance_totale_adj(G_drive_service, route_drive_service)
print("Distance totale (via G.adj) :", distance_adj, "mètres")
```

Affichage de la console :

```
Distance totale (méthode GPS) : 1562.7497744664697 mètres
Distance totale (via G.adj) : 1572.805 mètres
```

C-8-1 (/1pt) Calculer le temps de parcours sachant que la limitation de vitesse est 50 km/h en ville pour les véhicules et que la vitesse moyenne d'un vélo est de 20 km/h.

Implémentation d'une fonction temps_parcours :

```python
def temps_parcours(distance_m, vitesse_kmh):
    vitesse_ms = vitesse_kmh * 1000 / 3600
    temps_s = distance_m / vitesse_ms
    minutes = int(temps_s // 60)
    secondes = int(temps_s % 60)
    return minutes, secondes
```

Contenu dans le main :

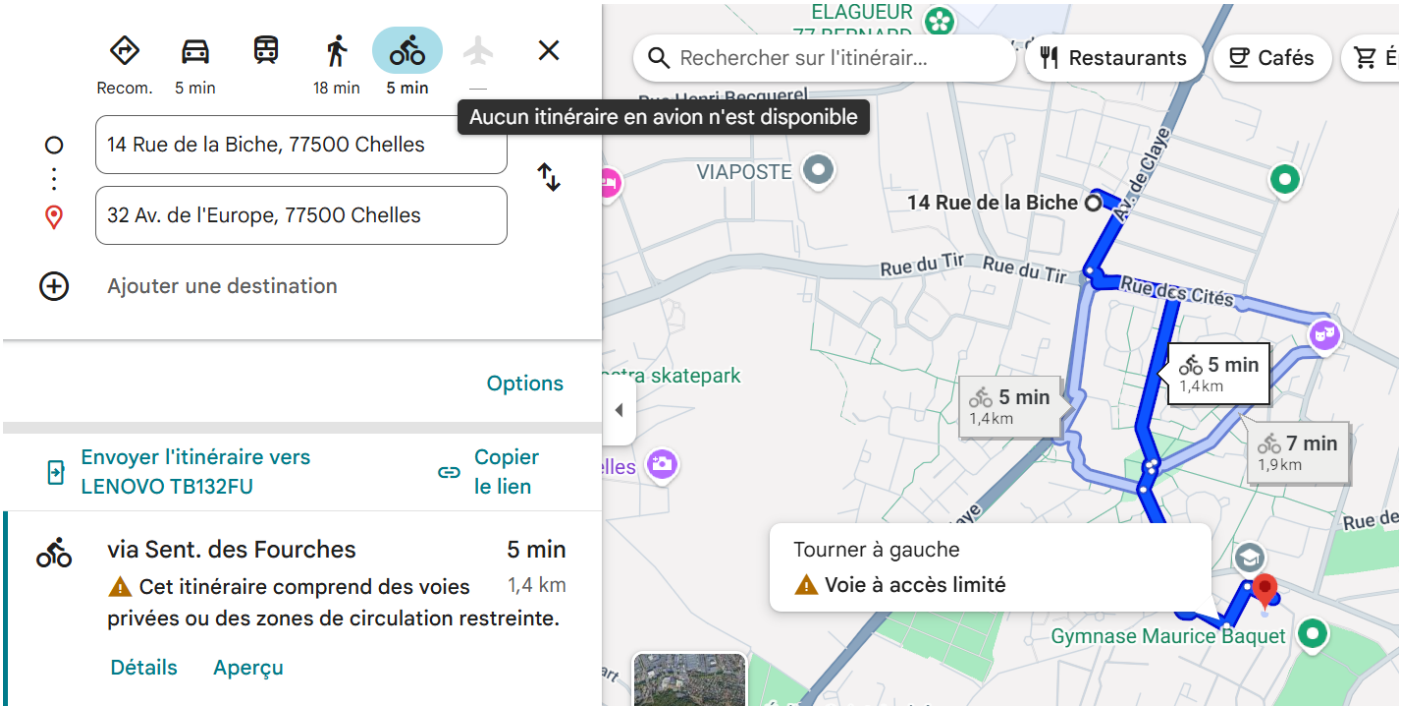
```python
G_drive, route_drive = graphe("14 rue de la biche", "Chelles", "32 avenue de l'Europe", "Chelles", "drive")
distance = distance_totale_adj(G_drive, route_drive)
print("Distance totale (via G.adj) :", distance, "mètres")
min_voiture, sec_voiture = temps_parcours(distance, 50)

min_velo, sec_velo = temps_parcours(distance, 20)

print("Voiture :", min_voiture, "minutes et", sec_voiture, "secondes")
print("Vélo :", min_velo, "minutes et", sec_velo, "secondes")
```

Affichage de la console :

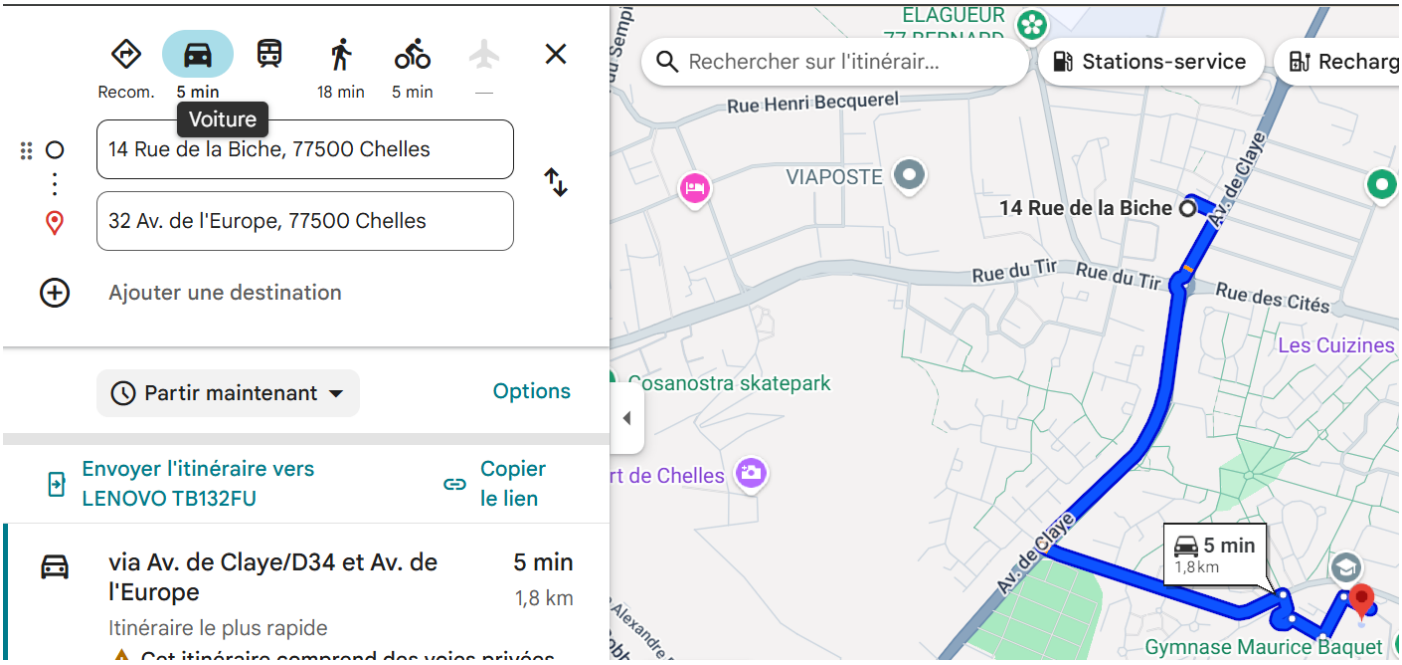
```
Distance totale (via G.adj) : 1682.823 mètres
Voiture : 2 minutes et 1 secondes
Vélo : 5 minutes et 2 secondes
>>>
```

C-8-2(/0.5pt) A l'aide de Google Maps, effectuer une copie d'écran d'un calcul d'itinéraires basés sur le parcours précédent en véhicule d'un graphe pondéré en temps. Comparer les itinéraires proposés et les temps théoriques et réels.

À vélo :



En voiture :


Recom.
5 min
Voiture
18 min
5 min
14 Rue de la Biche, 77500 Chelles
32 Av. de l'Europe, 77500 Chelles
Ajouter une destination
Partir maintenant
Options
Envoyer l'itinéraire vers LENOVO TB132FU
Copier le lien
via Av. de Claye/D34 et Av. de l'Europe
5 min
1,8 km
Itinéraire le plus rapide
Rechercher sur l'itinérair...
Stations-service
Recharg
ELAGUEUR
Rue Henri Becquerel
VIAPOSTE
14 Rue de la Biche
Av. de Claye
Rue du Tir
Rue du Tir
Rue des Cités
Les Cuizines
Cosanostra skatepark
rt de Chelles
5 min
1,8 km
Gymnase Maurice Baquet